

TEMA 02 - TypeScript

RA 2 [FRAMEWORKS] Y RA 7 [DWEK]

Parte 01

Ejercicios

Antes de empezar:

1. Dentro del repositorio `fw_ejercicios_apellido1_nombre1_25-26`

Usa la plantilla para crear la carpeta:

- `fw_t02p01_apellido1_nombre1`

EJERCICIO 1.

- AMPLIACIÓN SEGUNDA ORDINARIA -

Para superar la prueba de recuperación, será obligatorio que los distintos requisitos funcionales y las diferentes opciones del menú estén desarrollados y funcionando, al menos, al 50%. Si alguna de las partes principales de la práctica se deja sin desarrollar o presenta un grado de desarrollo insuficiente, la práctica se considerará no superada.

Antes de empezar: Todas estas ampliaciones y modificaciones deberán realizarse respetando los criterios seguidos durante el desarrollo de la práctica, especialmente en lo relativo al modelo de datos, la validación, la gestión de errores, la manipulación del DOM y la organización general de la aplicación.

Crea una nueva opción que se llame "Mis recetas"

Se incorporará una nueva opción en el menú principal que llevará a una nueva página llamada 'Mis recetas.'

Esta página deberá contener dos secciones claramente diferenciadas.

Sección 1. Listado de recetas creadas por el usuario

La primera sección mostrará un listado con todas las recetas creadas por el usuario autenticado en la aplicación.

Este listado se presentará en formato tabla e incluirá, al menos, la siguiente información:

identificador de la receta

nombre de la receta
categoría
país
número de ingredientes
número de imágenes asociadas

Cada receta tendrá asociada una opción del tipo "Ver detalles" y "Eliminar".

Al pulsar sobre "Ver detalles", se abrirá una ventana modal o un panel con toda la información completa de la receta, incluyendo sus ingredientes, cantidades, instrucciones e imágenes.

Al pulsar sobre "Eliminar" se pedirá confirmación y se eliminará esa receta si el usuario así lo desea.

Si el usuario no ha creado ninguna receta, deberá indicarse de forma clara en la interfaz.

Sección 2. Crear receta

La segunda sección permitirá crear una nueva receta propia del usuario.

La creación de la receta podrá hacerse de dos formas:

- crear una receta en blanco
- reutilizar una receta existente de la API

Opción A. Crear receta en blanco

En esta modalidad, el usuario deberá introducir manualmente toda la información necesaria de la receta.

Se solicitarán, al menos, los siguientes datos:

- nombre de la receta
- categoría
- país
- instrucciones
- ingredientes y cantidades
- una o varias imágenes asociadas a la receta

Los campos de categoría y país deberán introducirse mediante un select.

La aplicación deberá permitir asociar más de una imagen a la receta.

Las imágenes se incorporarán mediante una o varias URLs introducidas por el usuario. La aplicación deberá permitir su previsualización antes de guardar la receta.

La receta creada se almacenará en localStorage como una receta propia del usuario autenticado.

Opción B. Reutilizar una receta existente

En esta modalidad, deberá existir un buscador por nombre de receta que permita localizar recetas existentes.

Una vez seleccionada una receta, su información se utilizará como base para crear una nueva receta propia.

El usuario podrá modificar libremente los datos que considere necesarios antes de guardarla.

También en este caso se permitirá asociar más de una imagen a la receta mediante una o varias URLs introducidas por el usuario. La aplicación deberá permitir su previsualización antes de guardar la receta.

La nueva receta resultante se almacenará en localStorage como una receta propia del usuario autenticado y no deberá sustituir ni modificar la receta original obtenida desde la API.

Opción A y Opción B: Validaciones y almacenamiento

Se deberán aplicar las validaciones necesarias sobre los campos de la receta.

No se permitirá guardar una receta sin nombre, sin categoría, sin país o sin al menos un ingrediente informado correctamente.

Cada ingrediente deberá ir asociado a su cantidad correspondiente.

No se permitirá guardar una URL de imagen vacía, repetida o claramente inválida

Cada receta propia deberá tener un identificador único generado por la aplicación.

No se permitirá crear dos recetas propias del mismo usuario con el mismo nombre exacto.

Todas las recetas propias creadas en esta ampliación deberán almacenarse en localStorage, asociadas al usuario autenticado.

Normativa: La entrega se hará usando el repositorio compartido realizando un push con la solución final antes de la fecha límite de entrega. El mensaje del último commit debe ser: "Recuperación 2a. Ordinaria".

Crea una carpeta con nombre **ejercicio1_apellido1** dentro de **fw_t02p01_apellido1_nombre1**.

1. Dentro de esta carpeta crea un fichero [.gitignore](#) (preparado para typescript, vscode, logs, macos, windows y linux).

Asegúrate también de incluir /node_modules/, /vendor/ (si usas php) ...

Nota: Para facilitar la corrección de ejercicios vamos a mantener en el repositorio los ficheros js de dist. Por tanto, comenta estas líneas en .gitignore

```
# **/*.js
# **/*.js.map
# public/dist
# dist/
```

2. Dentro de esta carpeta crea un fichero readme.md (con la descripción del proyecto y versiones de las tecnologías)

3. Genera un archivo de configuración de TypeScript (tsconfig.json)

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES6",
    "rootDir": "./src/ts",
    "sourceMap": true,
    "outDir": "./public/dist",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true
  },
  "include": ["src/ts"],
  "exclude": ["node_modules"]
}
```

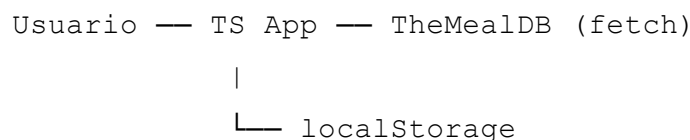
ESTRUCTURA DEL PROYECTO

```
/ejercicio1_apellido1
├── /src
│   ├── /...
│   ├── /ts                # Código TypeScript aquí
│   └── /ts/app.ts         # 'Orquestador' de nuestra aplicación
├── /public
│   ├── /dist              # Aquí se compilan los TS en JS
│   ├── /js                # Aquí irá JS externo o reutilizado
│   ├── /css
│   └── /img
```

```
|   |— index.html
|   |— ...
|— tsconfig.json
|— package.json
|— .gitignore
|— README.md
|— /node_modules
|— ...
```

En esta práctica vamos a crear una aplicación web utilizando TypeScript. Esta aplicación consume una API REST externa mediante AJAX moderno y hará uso del almacenamiento local del navegador, sin necesidad de implementar un servidor backend propio.

La aplicación estará basada en la API pública de recetas (TheMealDB), que devuelve información en formato JSON: <https://www.themealdb.com/api.php> La API requiere autenticación, pero en esta práctica se utilizará la API key de prueba "1", válida con fines educativos y que permite el acceso a las principales consultas.



Será necesario crear un sitio web de varias páginas haciendo uso de Bootstrap via CDN y su validación de formularios.

También se tiene que incluir una estructura básica de página formada por al menos:

- header con clases de Bootstrap
- navbar que nos permita navegar entre las distintas páginas (usando componentes de Bootstrap)
- main como contenedor principal
- footer también usando estilos de Bootstrap

El sitio web será adaptativo, al menos para un corte móvil y un corte de escritorio.

Muy importante: En una aplicación que consume una API externa no siempre todo funciona correctamente. La aplicación deberá gestionar y mostrar adecuadamente los siguientes tipos de errores:

- Errores de red (fetch lanza una excepción)
- Errores HTTP (la API responde, pero con error: 404, 500, etc.)
- Búsquedas sin resultados (la API devuelve un array vacío)
- Errores de uso por parte del usuario (input vacío)

En todos los casos se deberá mostrar un mensaje adecuado en la interfaz.

REQUISITOS:

La aplicación tiene una zona pública y una zona privada.

- La zona pública consta de:

- Una página index que muestra 8 recetas aleatorias de cualquier categoría. De una receta se muestra el nombre, la categoría, el país, la fotografía mediana y el número de ingredientes diferentes (en un badge).
 - Toda esta información se obtiene de la API.
 - Encima de las 8 imágenes habrá un select con todas las categorías ordenadas alfabéticamente (conseguidas de la API) donde la primera opción será "Todas las categorías". La primera opción estará marcada por defecto.
 - Si se cambian de opción se consiguen 8 recetas aleatorias de la categoría marcada.
 - No se pueden acceder a los detalles de una receta.
- Una página que nos permite iniciar sesión o crear un usuario.
 - Al crear un usuario se pide un nombre, un correo electrónico, una contraseña y una revalidación de contraseña.
 - El formulario valida que:
 - Los campos no esten vacios.
 - El correo electrónico sea un correo válido.
 - El correo electrónico no esté ya registrado en el sistema. Es decir sea único.
 - La contraseña tenga al menos 4 caracteres (para simplificar).
 - La revalidación de contraseña coincida con la contraseña y tenga al menos 4 caracteres
 - Cuando se crea se muestra un aviso descriptivo, pero no se inicia sesión automáticamente.

- Al iniciar sesión se pide un correo electrónico y una contraseña (y un check de mantener la sesión abierta o no - esta funcionalidad se deje para más adelante -). La contraseña no se puede recuperar.
 - El formulario valida que:
 - Los campos no esten vacios.
 - El correo electrónico sea un correo válido.
 - La contraseña tenga al menos 4 caracteres (para simplificar).
 - Si el usuario es válido se manda a la zona privada y se crea un usuario autenticado.
 - Si el usuario cierra el navegador y ha marcado mantener la sesión abierta, esta no se cerrara.
 - Por supuesto, después de iniciar sesión, no puedo acceder a la zona de crear un usuario y autenticarme.
- Toda esa información se almacena en localStorage.
- No es necesario gestionar la aceptación de cookies.
- No es necesario controlar si el usuario tiene bloqueado el uso de Javascript.

- La zona privada consta de:

- La página index de la zona pública con los siguiente elementos extra:
 - Botón para que el usuario logueado pueda guardar una categoría como favorita para que siempre empiece la aplicación seleccionando 8 recetas de esa categoría. Si no guarda ninguna categoría como favorita saldrá por defecto la opción "Todas las categorías". Toda esa información se almacena en localStorage.
 - Botón asociado a cada receta para que el usuario logueado pueda "Guardar" o "Des-guardar" una receta concreta. Toda esa información se almacena en localStorage.
 - El nombre de la receta y la imagen se convierten en un enlace a una página de más detalles.
 - Aparece una nueva sección con las 4 últimas recetas guardadas por el usuario identificado siguiendo el mismo estilo usado con las 8 recetas aleatorias. Si no hay ninguna receta guardada se informará de ello.
- No hay acceso a la página que nos permite iniciar sesión o crear un usuario. Se permite cerrar sesión al usuario.
- La página detalles de una receta muestra:
 - El nombre, la categoría, el país, la fotografía mediana, una lista ordenada de ingredientes y cantidades.
 - Botón "Guardar" o "Des-guardar".
 - Si la receta está guardada:
 - Un textarea para tomar anotaciones personales de la receta.
 - Opcional y con tamaño máximo de 250 caracteres.

- Un radio con las opciones: "Quiero hacerla" y "La he hecho".
 - Obligatorio y por defecto "Quiero hacerla".
- Valoración de 1 a 5.
 - Obligatorio si ha marcado "La he hecho".
- Fecha en la que se ha guardado.
- Botón "Guardar cambios".
- Botón "Cancelar".
- Toda esa información se almacena en localStorage.
- Si la receta se "Des-guardar" toda la información adicional, se elimina.
- La página plan semanal. Esta página tiene 2 secciones.
 - Sección 1. Crear plan semanal.
 - Un plan semanal tiene un nombre identificativo único de tipo cadena que se genera automáticamente y tiene el formato "YYYY-WXX" (donde YYYY es un año y XX es un número de semana dentro de ese año). Ejemplos: "2026-W01". Esto lo crea la aplicación cuando el usuario en un input obligatorio de tipo fecha selecciona una fecha concreta.
 - *Pídele a una IA que te ayude a generar la función que haga esto
(*function getISOWeek(date: Date): string*)
 - No puede haber dos planes con el mismo código. En ese caso el usuario deberá cambiar de semana.
 - Un plan semanal va de lunes a domingo.
 - Un plan semanal almacena (para cada día) solo la receta para la comida y la receta para la cena. No es necesario que las recetas del plan semanal estén guardadas por el usuario.
 - Un plan semanal debe tener al menos una receta en al menos uno de los días de forma obligatoria.
 - Un plan semanal tiene un buscador de recetas por un ingrediente. De entre los resultados de búsqueda se elige la comida o cena para cada día.
 - Botón "Guardar cambios".
 - Botón "Cancelar".
 - Toda esa información se almacena en localStorage.
- Sección 2. Tabla con planes semanales.
 - Aparece una tabla con los planes semana del usuario.
 - Los planes semanales en la tabla estarán ordenados de fecha de semana mayor a fecha de semana menor (usa ordenación alfabético inverso).
 - El plan semanal de la semana actual aparecerá destacado.
 - Cada plan semanal tendrá asociado un botón borrar que nos permitirá borrar el plan semanal de un usuario (tras confirmación).
 - El plan mostrará el nombre identificativo del plan (YYYY-WXX), el nombre de receta de la comida del lunes (que será un enlace ver más detalles) y una

imagen en miniatura, el nombre de la receta de la cena del lunes (que será un enlace ver más detalles) y una imagen en miniatura (voluntario), el nombre de la receta del martes (que será un enlace ver más detalles) y una imagen en miniatura, etc.

Funcionalidad Siguiendo lo exigido en el enunciado	¿Obligatorio?	Puntos
Index - Recetas aleatorias + Select	Obli.	2,00
Index Auth - Recetas Guardadas	Obli.	1,00
Detalles Auth - Detalles receta + Receta guardada	Obli.	3,00
*Planes Auth - Planes semanales: crear + ver	Rec.	2,50
*Se implementará solo en Angular. Ahora solo se plantea como se propondría la solución a este problema.		
Caché recetas	Vol.	0,50
Autenticación de usuario	Vol.	1,00

Arranque de la aplicación: app.ts

De forma recomendada vamos a tener un archivo principal TypeScript (llámalo app.ts o index.js) que actúe como orquestador de tu aplicación: inicializa servicios, configura el DOM, registra event listeners, etc.

Clases/Interfaces interesantes a implementar:

Clases → todo lo que creas, modificas, guardas, tiene métodos.
Interfaces → todo lo que solo describe la forma de los datos, normalmente para tipar objetos que vienen de la API o que se guardan/leen en localStorage sin comportamiento adicional.

Todas las clases:

- Podrán ampliarse o adaptarse si el alumno lo considera necesario.
- Deberán definir los métodos que consideren necesarios para su correcto funcionamiento.

Los datos se persistirán mediante localStorage.

1. Interfaz User: Representa un usuario registrado en la aplicación.

Atributos mínimos:

id: number (incremental)
name: string
email: string
password: string (en texto plano y en localStorage **solo con fines educativos**)
favoriteCategory?: string

2. Clase AuthSession (aunque también puede ser una interfaz): Representa la sesión del usuario autenticado. Clase muy simple, también podría ser una interfaz.

Atributos mínimos:

userId: number
name: string
loginDate: Date
¿NO/SÍ cerrar sesión? Pensarlo.

3. Interfaz UserMeal: Representa la relación entre un usuario y una receta guardada. Si una receta no está en UserMeal, el usuario no la ha guardado.

Atributos mínimos:

userId: number
mealId: number

*id debe coincidir exactamente con el idMeal de la API

saveDate: Date
status: enum - "QUIERO_HACERLA", "LA_HE_HECHO"
notes?: string
rating?: number

4. Interfaz WeeklyPlan: Representa un plan semanal de un usuario.

Atributos mínimos:

id: string (formato YYYY-WXX)
userId: number
days: WeeklyPlanDay[]

5. Interfaz WeeklyPlanDay: Representa un día concreto dentro de un plan semanal.

Atributos mínimos:

day: string (lunes, martes, etc.)

lunchMealId?: number
dinnerMealId?: number

*id debe coincidir exactamente con el idMeal de la API

6. Interfaz UserMiniMeal: Representa la mínima información de una receta obtenida de la API en algún momento y que el usuario ha usado en un plan semanal (no es necesario que las recetas del plan semanal estén guardadas por ese usuario). Así evitamos tener que estar haciendo peticiones constantes a la API en la sección de planes semanales. Es una especie de caché. Cuando se borra un plan semanal, no es necesario refrescar la caché.

Atributos mínimos:

id: number
name: string
image_small: string

*id debe coincidir exactamente con el idMeal de la API

Si falta información en algún momento, se consulta a la API y se cachea.

7. Interfaz MyMeal: Representa la información de una receta que la aplicación necesita mostrar en la interfaz de usuario. Esta interfaz no coincide con el JSON completo devuelto por la API, sino que es una versión reducida y adaptada a las necesidades de la aplicación. Se utilizará para transformar los datos recibidos de la API y evitar trabajar directamente con estructuras externas complejas. **No se almacena en localStorage.**

Atributos mínimos:

idMeal: number
strMeal: string
 //nombre de la receta
strCategory: string
strArea: string
strMealThumb: string
 //imagen tamaño mediana
ingredients: array de objetos con:
 name: string
 measure: string

(Los ingredientes deben ir ordenados alfabéticamente por nombre). Puedes crear una interfaz el par ingrediente medida.

El JSON recibido de la API debe transformarse a un objeto que cumpla la interfaz MyMeal antes de ser utilizado por la aplicación.

8. Clase ApiService: centraliza todas las peticiones a TheMealDB. Esta clase es la única que usa fetch. El resto de la aplicación consume Promesas, pero no hace fetch.

Atributos mínimos:

API_URL: string

API_KEY: string

Responsabilidades

Obtener recetas aleatorias (con o sin categoría)

Obtener recetas por ingrediente

Obtener detalles completos de una receta

Obtener categorías disponibles

...

Todas las funciones: Son asíncronas y devuelven datos en formato JSON o modelos internos. Nunca toca el DOM.

9. Clase StorageService: gestiona el acceso a localStorage.

Atributos mínimos (voluntario):

USER_KEY_ITEM, USER_MEAL_KEY_ITEM, ...

Responsabilidades

Alta y validación de usuarios

Gestión de sesión

Guardar y recuperar recetas del usuario

Guardar y recuperar planes semanales

Guardar preferencias del usuario

...

Nunca toca el DOM

10. Clase ViewService: pinta la interfaz (vengan los datos de API o de localStorage)

Responsabilidades

Renderizar listados de recetas

Renderizar detalles de receta

Renderizar planes semanales

Mostrar mensajes de error o aviso

...

Las funciones siempre reciben el elemento contenedor del DOM y los datos a representar.

11. Clase Utilities: Clase de apoyo con atributos y funciones estáticas comunes a toda la aplicación. Serán funciones de **lógica PURA**.

Responsabilidades:

Valores por defecto de la aplicación

Conversión de fechas a formato YYYY-WXX

Validación de formularios

...

Import/Export

Aviso: Cuando se utiliza TypeScript sin herramientas adicionales como Node.js, Webpack o Vite, se hace uso de módulos ES nativos del navegador.

Esto implica que los ficheros JavaScript generados deben cargarse usando:

```
<script type="module" src="./dist/index.js"></script>
```

Además, ES MUY IMPORTANTE que todas las sentencias import usadas en los ficheros .ts incluyan explícitamente la extensión .js, ya que el navegador no resuelve módulos como lo hace TypeScript o VS Code durante el desarrollo.

```
import { suma } from "./suma.js";
```

Asimismo, en aplicaciones con módulos ES, solo se carga en el HTML el módulo principal. El resto de ficheros JavaScript se cargan automáticamente mediante import. Es decir, no es necesario que estén todos referenciados en el html usando la etiqueta script.

Fichero: user.ts:

```
export interface User {  
  id: number;  
  name: string;  
  email: string;  
  password: string; // Texto plano (solo educativo)  
  favoriteCategory?: string;  
}
```

Fichero app.ts:

```
import { User } from "./user.js"; //Cuidado: vscode no añade el .js  
...
```

```
const testUsers: User[] = [...];
...

Ficheros .html:
...
<script type="module" src="dist/user.js"></script>

<!-- Todos los js se pueden obviar menos el JS principal -->
<script type="module" src="dist/app.js"></script>
```

Relación general entre clases:

- User → tiene muchos UserMeal
- User → tiene muchos WeeklyPlan
- WeeklyPlan → tiene varios WeeklyPlanDay
- WeeklyPlanDay → referencia a UserMiniMeal (y si necesita más información al Meal de la API).

Estructura recomendada mínima de claves en localStorage:

- Usuarios: users
 - Contenido: Array de User
- Sesión activa: authSession (en un futuro puede ser un sessionStorage)
 - Contenido: AuthSession
- Recetas guardadas por usuario: userMeals_<userId>
Ejemplo: userMeals_3
 - Contenido: Array de UserMeal (solo de ese usuario)
- Planes semanales por usuario: weeklyPlans_<userId>
 - Contenido: Array de WeeklyPlan
- Recetas mínimas cacheadas: userMiniMeal_<userId>
 - Contenido: Array de UserMiniMeal.

¿Qué NO se guarda en localStorage?

- Categorías
- Áreas
- Ingredientes
- Recetas **completas**

Solo se guarda IDs de recetas, la información mínima y datos añadidos por el usuario

Clave: users

```
[
  {
    "id": 56,
    "name": "Monkey D. Luffy",
    "email": "luffy@educastillalamancha.es",
    "password": "Mluffy",
    "favoriteCategory": "Seafood"
  },
  {
    "id": 60,
    "name": "Roronoa Zoro",
    "email": "zoro@educastillalamancha.es",
    "password": "roronoaR",
    "favoriteCategory": "Beef"
  }
]
```

Clave: authSession

```
{
  "userId": 56,
  "name": "Monkey D. Luffy",
  "loginDate": "2026-01-15T20:15:00.000Z"
}
```

Clave: userMeals_56

```
[
  {
    "userId": 56,
    "mealId": 52772,
    "saveDate": "2026-01-15T20:20:00.000Z",
    "status": "LA_HE_HECHO",
  }
]
```

```

    "notes": "Muy fácil de preparar. Añadí un poco más de ajo.",
    "rating": 5,
  },
  {
    "userId": 56,
    "mealId": 52819,
    "saveDate": "2026-01-15T21:21:00.000Z",
    "status": "QUIERO_HACERLA"
  },
  {
    "userId": 56,
    "mealId": 52944,
    "saveDate": "2026-01-15T22:22:00.000Z",
    "status": "QUIERO_HACERLA"
  },
]

```

Clave: userMiniMeals_56

```

[
  {
    "id": 52772,
    "name": "Teriyaki Chicken Casserole",
    "image_small": "https://www...wvpsxxl468256321.jpg"
  },
  {
    "id": 52819,
    "name": "Grilled Salmon",
    "image_small": "https://www...1548772327.jpg"
  },
  {
    "id": 52944,
    "name": "Seafood Pasta",
    "image_small": "https://www...xxxyyy.jpg"
  },
  {
    "id": 53026,
    "name": "Spicy Beef Bowl",
    "image_small": "https://www...zzzwww.jpg"
  }
]

```

Clave: weeklyPlans_56

```

[
  {
    "id": "2026-W02",
    "userId": 56,

```



```
"days": [
  {
    "day": "lunes"
  },
  {
    "day": "martes"
  },
  {
    "day": "miércoles"
  },
  {
    "day": "jueves"
  },
  {
    "day": "viernes"
  },
  {
    "day": "sábado"
  },
  {
    "day": "domingo",
    "dinnerMealId": 52819
  }
],
{
  "id": "2026-W03",
  "userId": 56,
  "days": [
    {
      "day": "lunes",
      "lunchMealId": 52772,
      "dinnerMealId": 52819
    },
    {
      "day": "martes",
      "lunchMealId": 52944,
      "dinnerMealId": 53026
    },
    {
      "day": "miércoles"
    },
    {
      "day": "jueves"
    },
    {
      "day": "viernes"
    },
    {
      "day": "sábado"
    }
  ]
}
```

```
    },  
    {  
      "day": "domingo"  
    }  
  ]  
}  
]
```

Consideraciones clave de TypeScript

Activar modo estricto (strict: true)

Activa todas las comprobaciones estrictas de TypeScript en el tsconfig.json. Esto detecta errores comunes como variables no inicializadas, valores null sin comprobar, o parámetros implícitamente any.

Evitar any a toda costa

El tipo any desactiva completamente TypeScript y te devuelve a JavaScript sin protección. Si no sabes el tipo exacto, usa unknown (que te obliga a validar antes de usar) o define una interface específica.

Tipar SIEMPRE parámetros y retornos de funciones

Aunque TypeScript puede inferir tipos en variables, NUNCA confíes en la inferencia para funciones. Especifica explícitamente qué recibe y qué devuelve cada función. Esto documenta tu código y previene errores.

Tipar arrays específicamente

No uses Array genérico. Especifica qué contiene: User[] o Array<User>. Esto permite autocomplete y detecta si intentas meter datos incorrectos en el array.

Union types para valores múltiples posibles

Cuando una variable puede ser de varios tipos específicos, usa union types (string | number). Es mejor que any porque limitas las opciones y mantienes el control.

Usar interface para estructuras de datos

Las interfaces definen la "forma" de los objetos. Son perfectas para modelos de datos (User, Meal, Plan...). Son más ligeras que las clases y desaparecen en el JavaScript compilado.

Usar class solo cuando hay comportamiento

Reserva las clases para cuando necesites métodos con lógica (servicios, utilidades). Si solo necesitas almacenar datos, una interface es suficiente y más eficiente.



Readonly para proteger datos inmutables

Marca propiedades como readonly cuando no deban cambiar después de la inicialización. Previene modificaciones accidentales y hace el código más predecible.

Aprovechar enum para valores constantes

Evita errores de tipeo, mejora el autocomplete y hace el código más mantenible.

Null safety con operadores modernos

Usa el operador optional chaining (?.) para acceder a propiedades que pueden no existir y el nullish coalescing (??) para valores por defecto. Evita largas cadenas de comprobaciones if

Literal types para valores exactos

Puedes especificar que una variable solo acepta valores concretos ("lunes" | "martes" | "miércoles"). Es más restrictivo que string y previene errores de valores inválidos.

No mezclar lógica de negocio con manipulación del DOM

Los servicios (ApiService, StorageService) NUNCA deben tocar el DOM. Solo ViewService manipula HTML. Esta separación es crucial para migrar a Angular después.

Gestionar errores

En los catch, el error es unknown, no any. Valida su tipo antes de usarlo.

Aspectos que ya veníamos haciendo y vamos a mantener:

Preferir const sobre let, nunca var

Usa const por defecto para inmutabilidad. Solo usa let cuando realmente necesites reasignar.

Usar async/await en lugar de Promises encadenadas

Las promesas con .then() son difíciles de leer y debuggear. Con async/await el código asíncrono se lee como código síncrono y es más fácil manejar errores con try-catch.

Separar código en módulos (un archivo = una responsabilidad)

No pongas todo en un único archivo.

Nombrar archivos según su contenido

Los nombres de archivo deben reflejar exactamente qué contienen: User.ts para la interface User, StorageService.ts para la clase StorageService. Facilita encontrar código.

Validar datos externos (API, localStorage)

TypeScript solo valida en tiempo de compilación. Los datos de APIs o localStorage pueden ser cualquier cosa en runtime. Siempre valida antes de usar con guardas de tipo.

Pensar en reutilización desde el principio

Cada función debe hacer UNA cosa y hacerla bien. Si una función mezcla responsabilidades, será difícil reutilizarla en Angular.

- Migración a Angular -

- ApiService → Se convierte en un @Injectable() Angular service casi sin cambios
- StorageService → Igual, @Injectable() service
- ViewService → Se elimina, pasa a ser componentes Angular
- Clases de modelo → Se convierten en interfaces TypeScript

.gitignore

```
# Archivos macOS
.DS_Store
.AppleDouble
.LSOVERRIDE
Icon?
.*
.Trashes
.fsevents
.Spotlight-V100

# Archivos Windows
Thumbs.db
ehthumbs.db
Desktop.ini
$RECYCLE.BIN/
*.lnk
*.lock
*.temp
*.bak
*.old

# Archivos Linux
*~
.directory
.Trash-*

# Archivos generados por VS Code
.vscode/
*.code-workspace

# Dependencias de Node.js (instaladas con npm install)
node_modules/
package-lock.json
```

```
pnpm-lock.yaml
yarn.lock

# Archivos generados por TypeScript
# **/*.js      En nuestro caso el contenido de este directorio lo mantenemos
# **/*.js.map  En nuestro caso el contenido de este directorio lo mantenemos
# **/*.d.ts
# public/dist  En nuestro caso el contenido de este directorio lo mantenemos
# dist/        En nuestro caso el contenido de este directorio lo mantenemos
build/
out/
compiled/
*.tsbuildinfo

# Archivos logs
logs/
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Archivos de compilación y caché
.cache/
out/
temp/
tmp/
*.swp
*.bak
*.tmp

# Coverage / tests
coverage/

# Archivos sensibles
.env
.env.local
.env.*
```